

Invoice Workbench AWS Deployment Guide

A complete, beginner-friendly walkthrough for hosting the Invoice Workbench application on Amazon Web Services

EC2

Elastic IP

Route 53

SSL / HTTPS

Application: Invoice Workbench — Split, Compare & Revise Invoice Documents

Backend: FastAPI (Python) • **Frontend:** HTML, CSS, JavaScript

Prepared for: Managers, Clients, Business Users & Developers

Document type: Deployment Guide, Knowledge Transfer & Client Reference

Table of Contents

1. Introduction	3
2. Client Prerequisites	5
3. AWS Services Overview	7
4. AWS Login Process	9
5. AWS Console Overview	10
6. Creating the EC2 Instance	11
7. Security Groups	14
8. Creating a Key Pair	15
9. Connecting to EC2	16
10. Deploying Invoice Workbench	17
11. Testing the Application	18
12. Elastic IP	19
13. Route 53	20
14. Existing Domain vs. New Domain	22
15. SSL / HTTPS	23
16. AWS Certificate Manager	24
17. Final Architecture	25
18. Validation Checklist	26
19. Common Issues & Troubleshooting	27
20. Deployment Checklist	28
Appendix A. Glossary of AWS Terms	29
21. Executive Summary (For Managers)	30

HOW TO USE THIS GUIDE

Sections 1–5 build essential background knowledge. Sections 6–17 are the hands-on, step-by-step deployment walkthrough. Sections 18–20 support go-live validation and handoff. Appendix A is a quick-reference glossary, and Section 21 is a one-page summary written for non-technical stakeholders.

Introduction

What is AWS?

AWS (Amazon Web Services) is a collection of computers, owned and maintained by Amazon, that anyone can "rent" over the internet. Instead of buying and maintaining a physical computer in an office, you rent computing power, storage, and networking from Amazon's data centers — and you only pay for what you use.

WHY THIS MATTERS

Think of AWS like renting an apartment instead of building a house. You get a fully working space immediately, someone else maintains the building (electricity, security, plumbing), and you can move to a bigger or smaller space whenever you need to.

What is Cloud Hosting?

Cloud hosting simply means running your application on a computer that lives in a data center (the "cloud") instead of on a laptop or an office desktop. That remote computer is always switched on, always connected to the internet, and can be reached by anyone, anywhere, at any time.

Why Do We Need AWS?

Invoice Workbench currently runs only on a developer's personal laptop. This works for testing, but it creates real business problems:

- The application is only available while that specific laptop is switched on and connected to the internet.
- Managers and clients cannot open the tool from their own computers or phones.
- There is no professional web address (URL) to share with stakeholders.
- There is no security certificate, so data would not be encrypted in transit.

Hosting Invoice Workbench on AWS solves all four problems with a small monthly cost and a one-time setup effort.

Current State vs. Future State

CURRENT STATE (Today)



Only reachable while the laptop is on and connected — not accessible to clients or managers.

FUTURE STATE (After this guide)

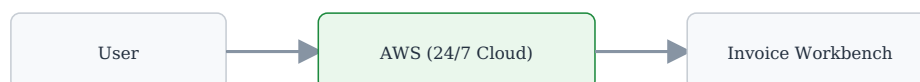


Figure 1.1 — Moving from a laptop-hosted tool to a 24/7 cloud-hosted service.

Benefits of Hosting Invoice Workbench on AWS

Benefit	What it means for the business
Always available	The application runs 24/7, not just while a laptop is switched on.
Professional web address	Clients and managers open a simple link instead of asking a developer to start the app.
Secure (HTTPS)	Invoice data is encrypted while travelling over the internet.
Scalable	If more people start using it, AWS resources can be increased without rebuilding anything.
Centralized	Everyone uses the same live version — no more "which laptop has the latest copy?"

Client Prerequisites

Before deployment can begin, we need two things from the client. This section explains exactly what we are asking for, and why.

REQUIREMENT 1 OF 2

Please provide AWS account access with permissions to create and manage AWS resources.

Why we need this: Every AWS server, IP address, and domain setting lives inside an AWS account. We cannot create anything without permission to work inside that account.

What is an "IAM user"? IAM stands for Identity and Access Management. Instead of sharing the master account password (called the "root" login), AWS lets the account owner create a separate, restricted login called an **IAM user** — similar to giving a contractor a building access card instead of the master key.

BEGINNER TIP

Never share the AWS root password. Always create an IAM user with only the permissions needed for this project.

Sample credentials format we will need:

Field	Example value
Account ID	123456789012
IAM Username	invoiceworkbench-deploy
Console Sign-in URL	https://123456789012.signin.aws.amazon.com/console
Password	(sent separately/securely)

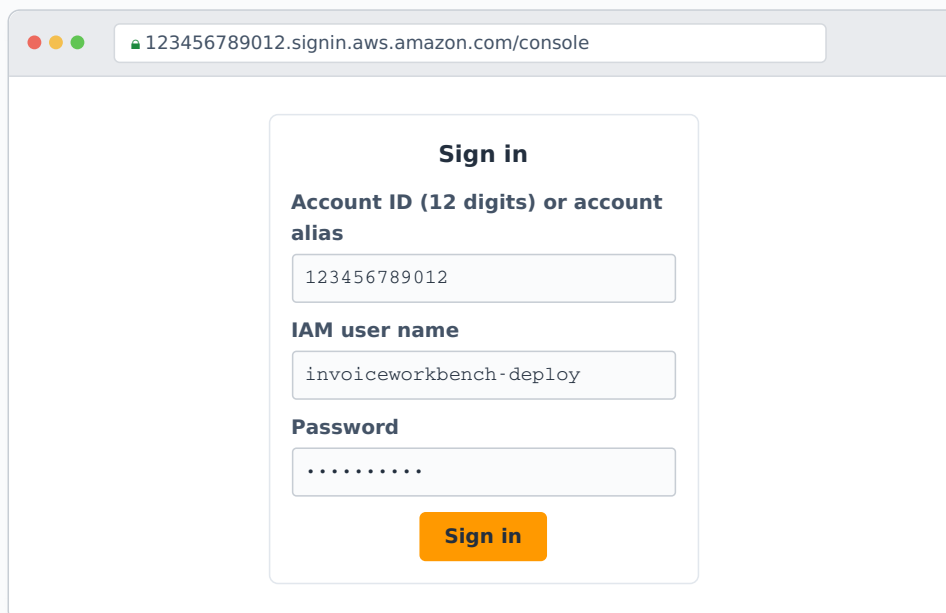
A screenshot of a web browser window showing the AWS IAM sign-in page. The browser's address bar displays the URL '123456789012.signin.aws.amazon.com/console'. The page has a white background with a central sign-in form. The form is titled 'Sign in' in bold. Below the title, it asks for the 'Account ID (12 digits) or account alias' and shows the example value '123456789012' in a text input field. Next, it asks for the 'IAM user name' and shows the example value 'invoiceworkbench-deploy' in a text input field. Then, it asks for the 'Password' and shows a masked input field with eight dots. At the bottom of the form is an orange 'Sign in' button.

Figure 2.1 — Illustrative AWS IAM sign-in screen.

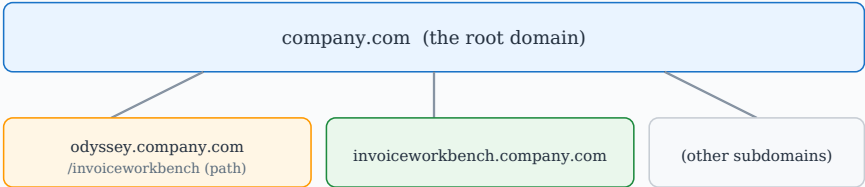
REQUIREMENT 2 OF 2

Please confirm the hosting URL/domain strategy for the Invoice Workbench utility.

A **domain name** is the readable web address people type into a browser (like *company.com*) instead of a hard-to-remember number. We need to know which of the following four options the client prefers:

Option	Example	Description
Existing Odyssey URL	https://odyssey.company.com/invoiceworkbench	Added as a page/path inside the existing Odyssey platform
Existing company domain	https://invoiceworkbench.company.com	A new subdomain added under the company's current domain
New subdomain	https://invoiceworkbench.company.com	Same as above — a dedicated address under the existing domain
New domain	https://invoiceworkbench.com	A brand-new domain purchased specifically for this tool

How Domains, Subdomains, and Paths Relate (DNS Explanation)



A subdomain (e.g. "invoiceworkbench.company.com") is a labeled branch of the main domain — it is free to create and needs no new domain purchase.

RECOMMENDATION

A new subdomain (e.g. *invoiceworkbench.company.com*) is usually the fastest and lowest-cost option because it reuses a domain the client already owns.

AWS Services Overview

Four AWS services work together to host Invoice Workbench. This section explains each one in plain language before we touch the AWS console. Keep this page handy — later sections refer back to these definitions.



Figure 3.1 — How the four services connect a user's browser to Invoice Workbench.

EC2 Elastic Compute Cloud (EC2)

WHAT IS IT?

A virtual computer ("server") that lives in an AWS data center. It behaves exactly like a physical computer — it has a processor, memory, storage, and an operating system — except it is rented by the hour or month.

WHY IS IT NEEDED?

Invoice Workbench's FastAPI backend and frontend files need somewhere to actually run, 24 hours a day. EC2 is that "always-on computer."

WHAT HAPPENS IF WE DON'T USE IT?

There is no server, which means the application code has nowhere to execute — it stays trapped on a developer's laptop.

INVOICE WORKBENCH EXAMPLE

We will launch one small EC2 instance running Ubuntu Linux, and install Python and the Invoice Workbench code on it.

ARCHITECTURE PLACEMENT

EC2 is the engine room — the last stop before "the app" itself in Figure 3.1.

EIP Elastic IP Address

WHAT IS IT?

A fixed, permanent internet address (like a phone number that never changes) that AWS assigns exclusively to our EC2 server.

WHY IS IT NEEDED?

By default, an EC2 server's public address can change every time it restarts. A fixed address means the domain name always points to the right place.

WHAT HAPPENS IF WE DON'T USE IT?

Every time the server restarts, the address changes, and the domain name would silently stop working until manually reconfigured.

INVOICE WORKBENCH EXAMPLE

We allocate one Elastic IP and permanently attach ("associate") it to the Invoice Workbench EC2 server.

AWS Services Overview

DNS Route 53 (Domain Name System)

WHAT IS IT?

AWS's phone-book service. It translates a human-readable web address (like `invoiceworkbench.company.com`) into the numeric Elastic IP address computers actually use.

WHY IS IT NEEDED?

People remember words, not numbers. Route 53 lets users type a friendly address instead of an IP address like `54.183.12.9`.

WHAT HAPPENS IF WE DON'T USE IT?

Users would have to type the raw IP address into their browser, which is unprofessional, hard to remember, and breaks if the IP ever changes.

INVOICE WORKBENCH EXAMPLE

We create a "Hosted Zone" for the domain and add an "A Record" pointing the chosen subdomain to the Elastic IP.

SSL / HTTPS Certificate

WHAT IS IT?

A digital "seal of trust" issued by a certificate authority that encrypts all data traveling between the user's browser and our server.

WHY IS IT NEEDED?

Invoice documents can contain sensitive financial data. HTTPS scrambles this data in transit so it cannot be read or altered by anyone intercepting it.

WHAT HAPPENS IF WE DON'T USE IT?

Browsers display a "Not Secure" warning, data travels unencrypted, and many corporate networks will block the site outright.

INVOICE WORKBENCH EXAMPLE

We request a free certificate through AWS Certificate Manager for the chosen domain, then attach it to the server.

Quick-Reference Summary

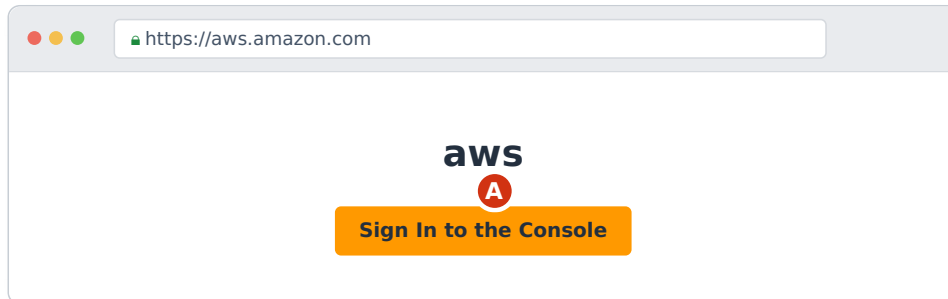
Service	Plain-language role	Invoice Workbench use
EC2	The rented computer that runs the app	Hosts FastAPI backend + frontend files
Elastic IP	A permanent address for that computer	Fixed IP so the domain never breaks
Route 53	Translates domain names to IP addresses	Points <code>invoiceworkbench.company.com</code> to the server
SSL/HTTPS	Encrypts data in transit, builds trust	Enables the padlock and <code>https://</code> prefix

AWS Login Process

Follow these five steps every time you need to access the AWS Console for Invoice Workbench.

1 Open the AWS Website

Open a browser and go to <https://aws.amazon.com>



A The orange "Sign In to the Console" button, top-right of the page.

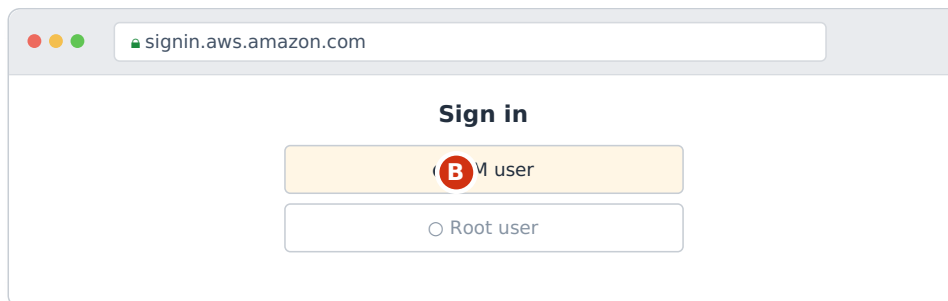
Figure 4.1 — Illustrative AWS homepage.

2 Click "Sign In to the Console"

This takes you to the account sign-in page.

3 Select "IAM user"

On the sign-in page, choose the **IAM user** option (not "Root user") and enter the 12-digit Account ID provided by the client.



B Choose "IAM user" — this is the restricted, safer login type.

Figure 4.2 — Illustrative sign-in type selector.

4 Enter Credentials

Type the IAM username and password supplied by the client (see Section 2, Requirement 1) and click **Sign in**.

5 Arrive at the AWS Console Home

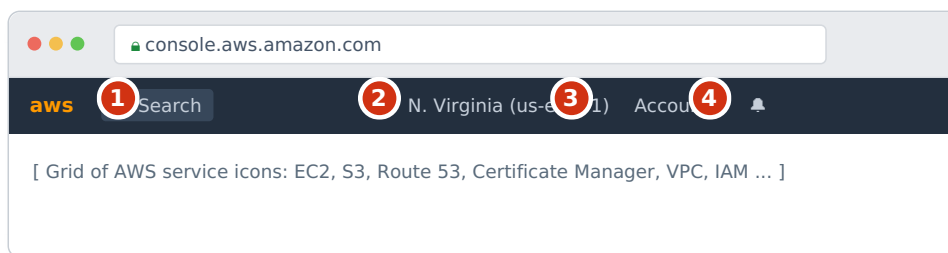
A successful login lands on the AWS Console Home page, which lists all available services.

EXPECTED RESULT

You should see the AWS Console Home screen with a search bar at the top and a grid of service icons. If you instead see an error message, double-check the Account ID, username, and password.

AWS Console Overview

Before creating anything, let's get familiar with the AWS Console layout. You will return to this same layout throughout the project.



- ① **Search bar** — type a service name (e.g. "EC2") to jump straight to it.
- ② **Region selector** — the geographic data center your resources live in. Keep this consistent for the whole project.
- ③ **Account menu** — billing, security settings, and sign-out.
- ④ **Notifications bell** — alerts about your account and resources.

Figure 5.1 — Illustrative AWS Console top navigation bar.

BEGINNER TIP

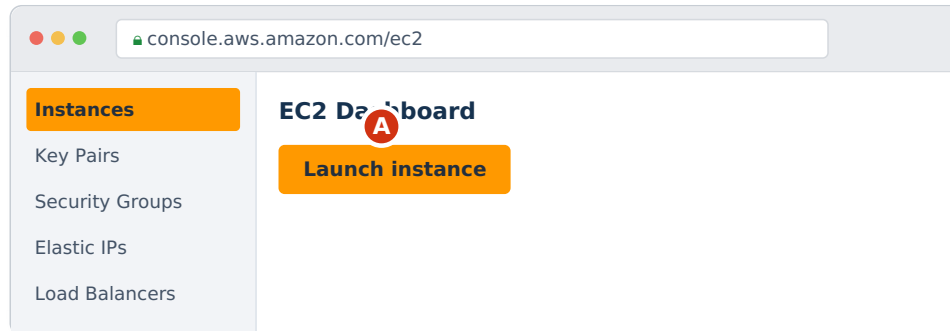
AWS Regions are separate physical data-center locations (e.g. North Virginia, Mumbai, London). Always create every resource for a single project in the **same region**, or they will not be able to find each other.

Creating the EC2 Instance

This is the most important step — we are creating the "computer" that will run Invoice Workbench.

1 Open EC2 and Click "Launch Instance"

Search for "EC2" in the console search bar, open the EC2 Dashboard, then click the orange **Launch Instance** button.



A Click "Launch instance" to start the setup wizard.

Figure 6.1 — Illustrative EC2 Dashboard.

2 Enter a Name

Give the server a clear, recognizable name.

Name

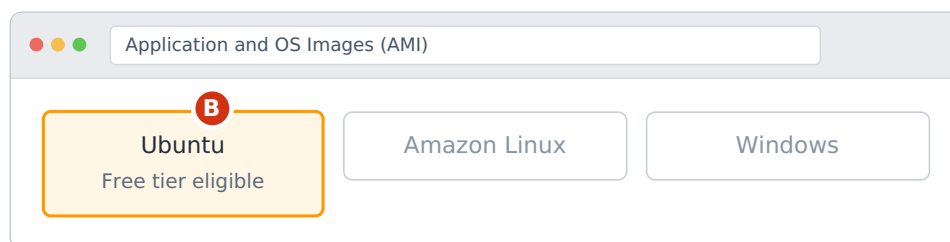
invoice-workbench-server

WHY

Clear naming avoids confusion later if the AWS account holds multiple servers.

3 Select "Ubuntu" as the Operating System (AMI)

An **AMI** (Amazon Machine Image) is a pre-packaged operating system template. Ubuntu is a free, beginner-friendly, widely supported Linux distribution — a good fit for running Python applications.



B Select the "Ubuntu" card.

Figure 6.2 — Illustrative operating system selection.

Creating the EC2 Instance

4 Select an Instance Type

The **instance type** determines how much processing power and memory the virtual computer has. For Invoice Workbench, a small, cost-efficient type is enough.

Instance type

t2.micro (1 vCPU, 1 GiB memory — Free Tier eligible)

BEGINNER TIP

Think of instance type like choosing a car engine size — bigger engines cost more to "fuel" (run) but handle heavier loads. Start small; AWS lets you upgrade later without losing your data.

5 Create a Key Pair

A **key pair** is a digital lock-and-key system used to securely connect to the server instead of a typed password. AWS keeps the "lock" (public key) on the server and gives you the "key" (private .pem file) to download once.

Key pair name

invoice-workbench-key

IMPORTANT

The .pem file can only be downloaded once, at creation time. Store it safely — without it, you cannot connect to the server over SSH.

6 Configure Networking

This step decides which "doors" (ports) into the server are open to the internet. We will configure this in detail in **Section 7 (Security Groups)**. For now, allow SSH traffic from your own IP address.

7 Configure Storage

This is the server's hard-drive size. The default of 8 GiB (gigabytes) is enough for Invoice Workbench's code and dependencies.

Storage

8 GiB gp3 (General Purpose SSD)

8 Review and Launch

Check all settings on the summary panel, then click **Launch instance**.

Launch an instance — Summary

Name	invoice-workbench-server
AMI	Ubuntu 22.04 LTS
Instance type	t2.micro
Key pair	invoice-workbench-key
Storage	8 GiB gp3

Launch instance

 Final confirmation button — this actually creates and starts billing for the server.

Figure 6.3 — Illustrative launch summary screen.

EXPECTED RESULT

The Instances list shows **invoice-workbench-server** with an Instance State of **Running** and a Status Check of **2/2 checks passed** within about a minute.

Security Groups

What is a Security Group?

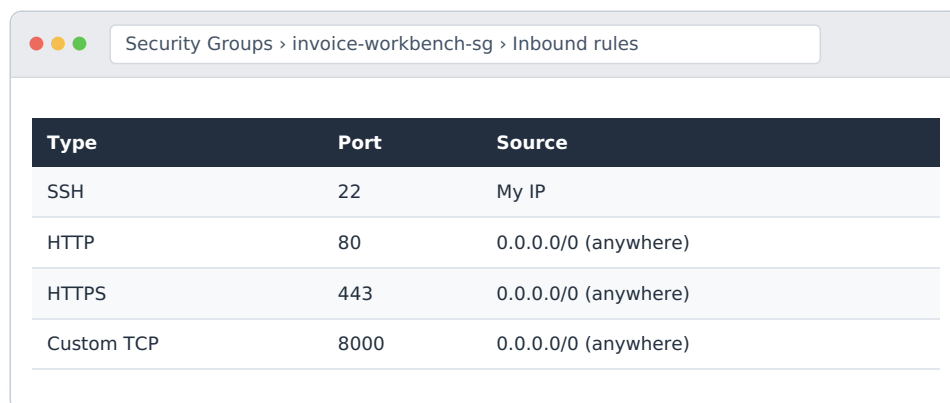
A **Security Group** is a virtual firewall attached to the EC2 server. It controls exactly which types of network traffic are allowed in ("inbound") and out ("outbound"). Think of it as a guest list at a building entrance — only pre-approved visitors (traffic types) are let through the door.

WHY IT'S NEEDED

Without a security group, the server would either be completely unreachable, or (if misconfigured) wide open to anyone on the internet. We open only the specific "doors" Invoice Workbench needs.

Ports Used by Invoice Workbench

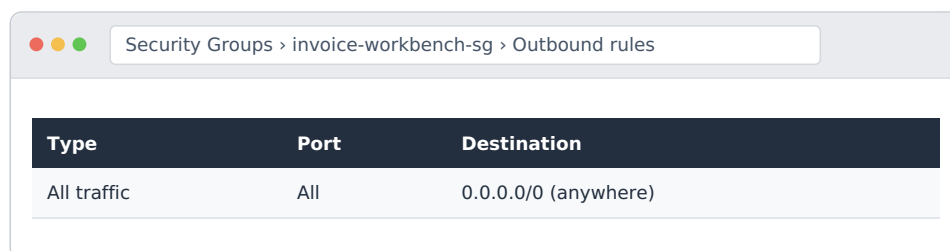
Port	Purpose	Plain-language meaning
22	SSH	Lets developers securely connect to the server's command line
80	HTTP	Standard unencrypted web traffic (redirects to HTTPS)
443	HTTPS	Encrypted web traffic — what users' browsers use
8000	FastAPI	The port the Invoice Workbench backend listens on directly



Security Groups > invoice-workbench-sg > Inbound rules

Type	Port	Source
SSH	22	My IP
HTTP	80	0.0.0.0/0 (anywhere)
HTTPS	443	0.0.0.0/0 (anywhere)
Custom TCP	8000	0.0.0.0/0 (anywhere)

Figure 7.1 — Illustrative Inbound Rules table.



Security Groups > invoice-workbench-sg > Outbound rules

Type	Port	Destination
All traffic	All	0.0.0.0/0 (anywhere)

Figure 7.2 — Illustrative Outbound Rules (default: allow all outgoing traffic).

BEGINNER TIP

Restricting SSH (port 22) to "My IP" instead of "anywhere" is one of the simplest ways to keep the server secure.

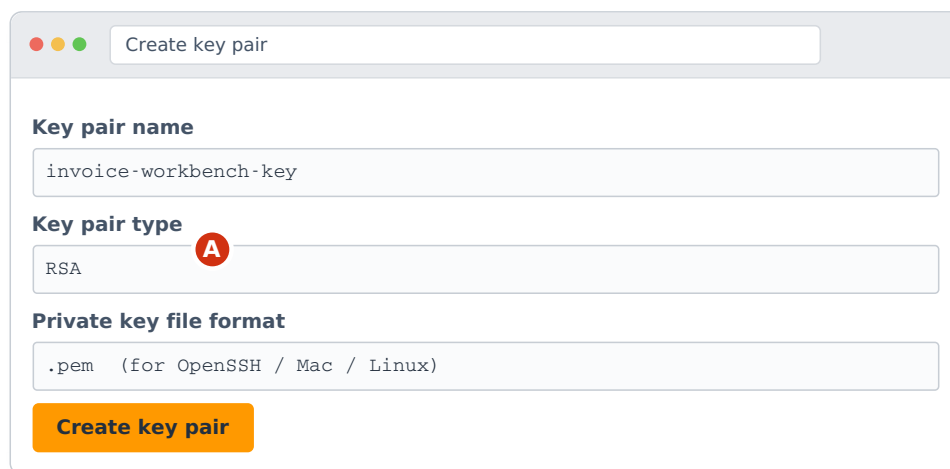
Creating a Key Pair

What is a Key Pair?

A key pair is made of two matching parts: a **public key** stored on the AWS server, and a **private key** (the .pem file) that you keep. Only someone holding the exact matching private key can open a secure connection to the server — similar to a lock that only one specific physical key can open.

Why It Is Important

- It replaces passwords for server access, which are easier to guess or steal.
- It is the **only** way to connect over SSH if EC2 Instance Connect (a browser-based option) is unavailable.
- If lost, there is no way to recover it — a new key pair and, in some cases, a new instance would be required.

A screenshot of the AWS Management Console 'Create key pair' dialog. The dialog has a title bar with three colored circles (red, yellow, green) and the text 'Create key pair'. Below the title bar, there are three sections: 'Key pair name' with a text input field containing 'invoice-workbench-key'; 'Key pair type' with a dropdown menu showing 'RSA' and a red circle with a white 'A' next to it; and 'Private key file format' with a dropdown menu showing '.pem (for OpenSSH / Mac / Linux)'. At the bottom of the dialog is an orange button labeled 'Create key pair'.

A Clicking this immediately downloads `invoice-workbench-key.pem` to your computer — save it somewhere safe.

Figure 8.1 — Illustrative Create Key Pair dialog.

IMPORTANT

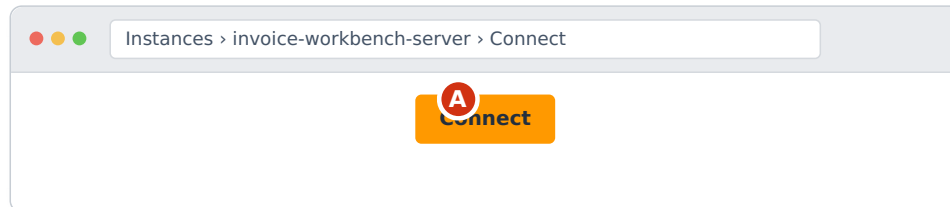
On Mac/Linux, run `chmod 400 invoice-workbench-key.pem` before using the file, or SSH will refuse to use it for security reasons.

Connecting to EC2

There are two ways to open a command line ("terminal") session on the server.

Option A — EC2 Instance Connect (Browser-Based)

The simplest option for beginners. AWS opens a terminal directly inside your browser — no extra software required.



A Click "Connect", choose the "EC2 Instance Connect" tab, then click "Connect" again.

Figure 9.1 — Illustrative Connect screen.

Option B — SSH from Your Own Terminal

For developers comfortable with the command line:

```
ssh -i invoice-workbench-key.pem ubuntu@PUBLIC-IP
```

EXPECTED SUCCESSFUL CONNECTION

```
ubuntu@ip-172-31-10-24:~$ _
```

A command prompt like the one above confirms you are now inside the server.

Deploying Invoice Workbench

Project Structure

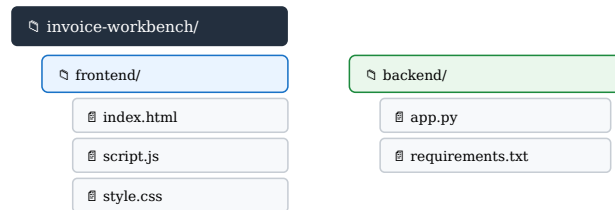


Figure 10.1 — Invoice Workbench folder structure.

1 Upload the Project Files

Use scp (secure copy) from your own computer to transfer the project folder to the server:

```
scp -i invoice-workbench-key.pem -r ./invoice-workbench ubuntu@PUBLIC-IP:~/
```

2 Install Python

```
sudo apt update && sudo apt install python3-pip python3-venv -y
```

WHY

Ubuntu servers do not always come with Python pre-installed. FastAPI needs Python 3 to run.

3 Install Dependencies

```
cd invoice-workbench/backend
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, and every other Python library Invoice Workbench needs.

4 Run the Application

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Uvicorn is the program that actually runs a FastAPI application and listens for incoming web requests. `--host 0.0.0.0` tells it to accept connections from any address (not just from inside the server itself), and `--port 8000` sets which "door" it listens on.

BEGINNER TIP

To keep the app running after you close the terminal, run it with a process manager (e.g. `tmux`, `screen`, or a `systemd` service) — otherwise it stops the moment the SSH session ends.

Testing the Application

With Uvicorn running, open a browser and go to:

```
http://PUBLIC-IP:8000
```

Replace **PUBLIC-IP** with the actual IP address shown on the EC2 Instances page.

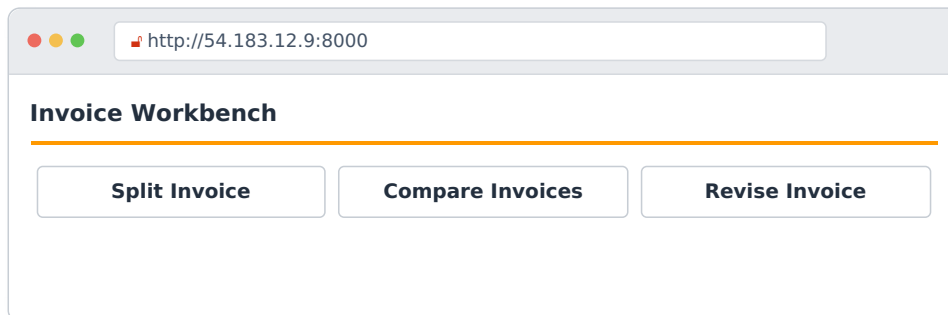


Figure 11.1 — Illustrative Invoice Workbench UI, reachable at the raw IP address (note the "Not Secure" state before HTTPS is configured in later sections).

SUCCESS CRITERIA

- The Invoice Workbench interface loads without errors.
- The Split, Compare, and Revise options are visible and clickable.
- No connection-refused or timeout errors appear.

Elastic IP

What is Elastic IP?

An Elastic IP is a public IP address that you reserve permanently and attach to your server. Unlike the default address EC2 assigns automatically, an Elastic IP does not change.

Why the Public IP Can Change

By default, AWS reassigns a new public IP address to an EC2 server every time it stops and starts again. This is fine for a server nobody depends on yet — but Invoice Workbench needs a stable address that a domain name can point to permanently.

Why Elastic IP Is Needed

Invoice Workbench example: once the Elastic IP (e.g. 54.183.12.9) is attached, the Route 53 domain record can safely point to that address forever, even if the server is restarted for maintenance.

Without Elastic IP

IP changes on restart

With Elastic IP

54.183.12.9 — permanent

1 Allocate Elastic IP

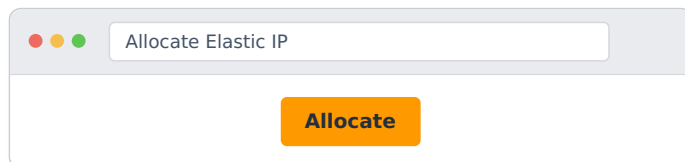


Figure 12.1 — Allocate screen.

2 Associate With Server

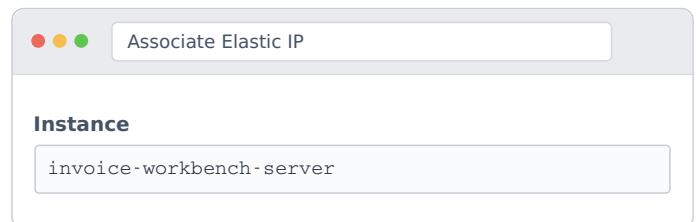


Figure 12.2 — Associate screen.

3 Verify

EXPECTED RESULT

The EC2 Instances page now shows the Elastic IP as the server's Public IPv4 address, and it stays the same after a restart.

Route 53

What is DNS?

DNS (Domain Name System) is the internet's phonebook. It matches human-friendly names (invoiceworkbench.company.com) to numeric addresses (54.183.12.9) that computers use to actually find each other.

What is Route 53?

Route 53 is Amazon's DNS service — the tool we use to create and manage that "phonebook entry" for Invoice Workbench.

Why Route 53 Is Required

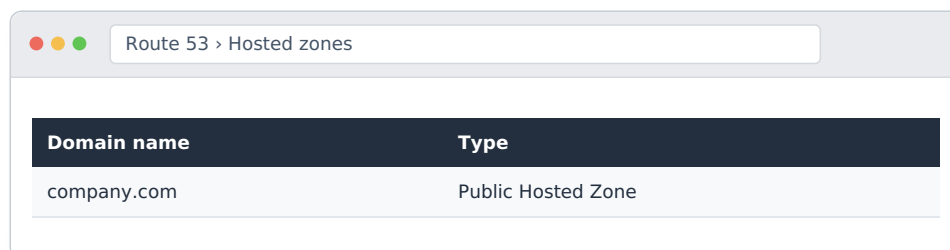
Without it, users would have to memorize and type a raw IP address, which is unprofessional and breaks if the address ever needs to change.

IP Address vs. Domain vs. Subdomain

Term	Example	Beginner-friendly meaning
IP Address	54.183.12.9	The server's actual "street address" on the internet
Domain	company.com	A readable name a business owns
Subdomain	invoiceworkbench.company.com	A named section under that domain, free to create

1 Hosted Zone

A **Hosted Zone** is the container in Route 53 that holds all DNS records for one domain.



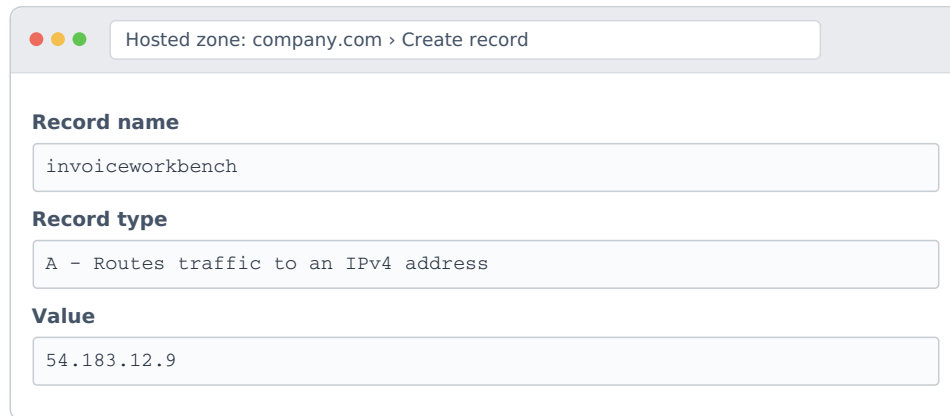
Domain name	Type
company.com	Public Hosted Zone

Figure 13.1 — Illustrative Hosted Zones list.

Route 53

2 DNS Records & A Record

An **A Record** is the specific instruction that says "this name points to this IP address."



The screenshot shows the AWS Route 53 console interface for creating a new record. At the top, a breadcrumb trail reads "Hosted zone: company.com > Create record". Below this, there are three input fields with labels: "Record name" containing "invoiceworkbench", "Record type" with a dropdown menu showing "A - Routes traffic to an IPv4 address", and "Value" containing the IP address "54.183.12.9".

Figure 13.2 — Illustrative A Record creation.

RECORD VALIDATION

After a few minutes, running `nslookup invoiceworkbench.company.com` should return the Elastic IP address, confirming the record is live.

Existing Domain vs. New Domain

Depending on the client's answer to Section 2, Requirement 2, deployment follows one of three scenarios.

1 Hosted Under Odyssey

Invoice Workbench is added as a path inside the existing Odyssey platform.

RESULT

```
https://odyssey.company.com/invoiceworkbench
```

2 New Subdomain

A dedicated subdomain is created under the client's existing domain (the recommended option).

RESULT

```
https://invoiceworkbench.company.com
```

3 New Domain

A brand-new domain is purchased and pointed at the Invoice Workbench server.

RESULT

```
https://invoiceworkbench.com
```

NOTE

All three scenarios use the exact same EC2 server, Elastic IP, and SSL certificate underneath — only the Route 53 record and certificate domain name differ.

SSL / HTTPS

What is SSL?

SSL (Secure Sockets Layer), and its modern replacement TLS, is a technology that encrypts data traveling between a browser and a server, so nobody in between can read or tamper with it.

What is HTTPS?

HTTPS is the secure version of HTTP (HyperText Transfer Protocol) — the language browsers and servers use to exchange web pages. The "S" means SSL/TLS encryption is switched on.

Why HTTPS Is Required

Invoice Workbench deals with financial documents. HTTPS ensures this data cannot be intercepted or modified while traveling across the internet, and it builds user trust through the browser's padlock icon.

HTTP vs. HTTPS

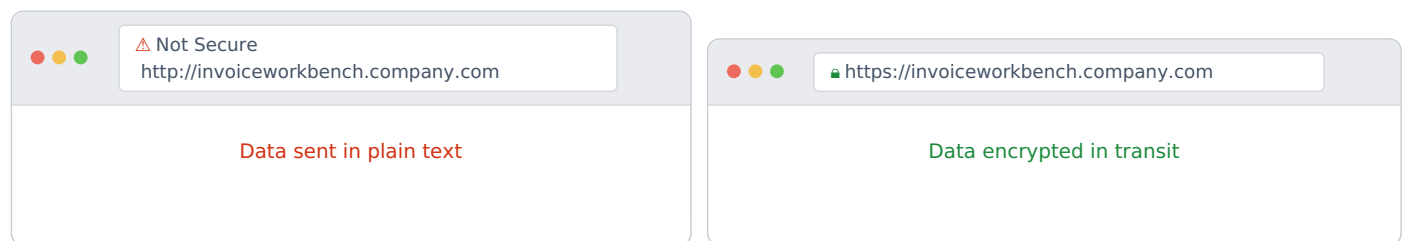
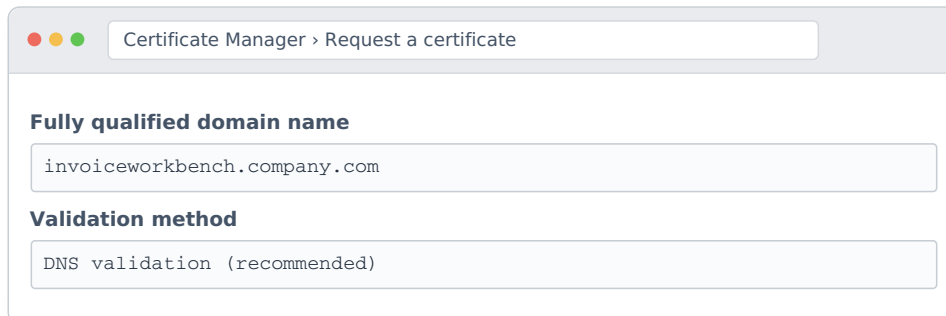


Figure 15.1 — Browser address bar before and after enabling HTTPS.

AWS Certificate Manager

AWS Certificate Manager (ACM) issues free SSL/TLS certificates for domains hosted in AWS.

1 Request a Certificate



Certificate Manager > Request a certificate

Fully qualified domain name

invoiceworkbench.company.com

Validation method

DNS validation (recommended)

Figure 16.1 — Illustrative certificate request screen.

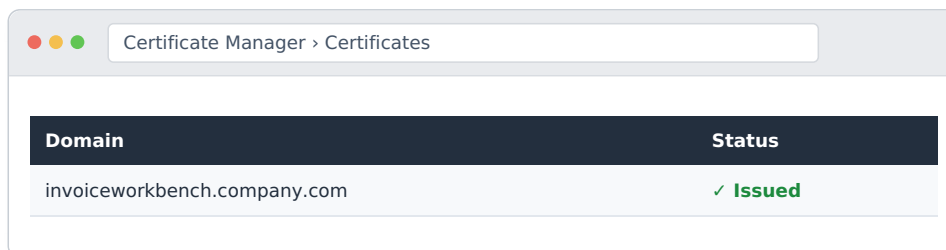
2 Domain Validation

ACM provides a special DNS record that proves you control the domain. We add this record inside the Route 53 Hosted Zone (Section 13).

WHY

This step stops anyone from requesting a certificate for a domain they don't own.

3 Certificate Issued



Domain	Status
invoiceworkbench.company.com	✓ Issued

Figure 16.2 — Illustrative issued certificate.

EXPECTED RESULT

Once issued, the certificate is attached to a load balancer or reverse proxy in front of the EC2 server so that HTTPS traffic is decrypted correctly.

Final Architecture

Here is the complete request flow once every section in this guide has been completed:

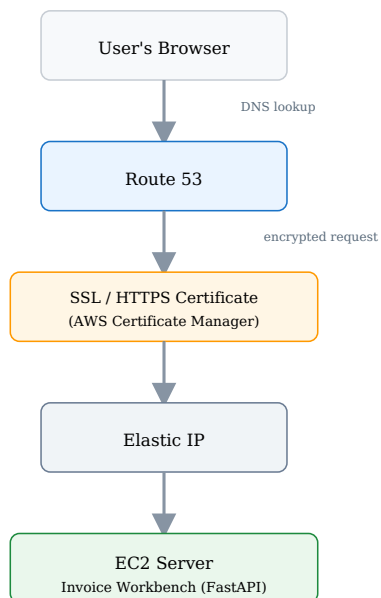


Figure 17.1 — Full request flow, from browser to application.

Request Flow Explained

1. The user types the domain name into their browser.
2. Route 53 resolves that name to the Elastic IP address.
3. The browser establishes a secure HTTPS connection, verified by the SSL certificate.
4. The encrypted request travels to the fixed Elastic IP.
5. The EC2 server receives the request and Invoice Workbench (running via Uvicorn) responds.

Validation Checklist

Use this checklist to confirm the deployment is fully working before handing off to the client.

- ✓ EC2 instance shows "Running" with 2/2 status checks passed
- ✓ Elastic IP is allocated and associated with the server
- ✓ DNS record resolves the domain to the Elastic IP (nslookup / dig)
- ✓ HTTPS loads with a valid padlock and no browser warnings
- ✓ Invoice Split feature works end-to-end
- ✓ Invoice Compare feature works end-to-end
- ✓ Invoice Revise feature works end-to-end

Common Issues & Troubleshooting

Symptom	Likely Cause	Fix
Browser can't reach the site at all	Security Group blocking the port	Add an inbound rule for the port being used (Section 7)
Domain doesn't resolve	DNS record missing or not yet propagated	Re-check the A Record in Route 53; wait up to 30 minutes
"Not Secure" warning in browser	Certificate not issued or not attached	Confirm ACM status is "Issued" and correctly linked (Section 16)
Application won't start	Missing dependency or Python error	Re-run <code>pip install -r requirements.txt</code> ; check terminal error output
Site loads on IP but not on domain	DNS points to the wrong IP or an outdated Elastic IP	Update the A Record to the current Elastic IP

BEGINNER TIP

When something doesn't work, always check these three things first, in order: (1) is the server running, (2) is the correct port open in the Security Group, and (3) does the domain resolve to the right IP.

Deployment Checklist

Pre-Deployment

- ✓ AWS account access confirmed (Section 2, Requirement 1)
- ✓ Domain/URL strategy confirmed (Section 2, Requirement 2)
- ✓ Invoice Workbench source code ready (frontend + backend)

Deployment

- ✓ EC2 instance launched (Section 6)
- ✓ Security Group configured (Section 7)
- ✓ Key pair created and secured (Section 8)
- ✓ Application files uploaded and dependencies installed (Section 10)
- ✓ Application tested on raw IP (Section 11)

Post-Deployment

- ✓ Elastic IP allocated and associated (Section 12)
- ✓ Route 53 hosted zone and A Record created (Section 13)
- ✓ SSL certificate requested, validated, and issued (Section 16)

Go-Live

- ✓ Final validation checklist completed (Section 18)
- ✓ URL shared with managers, clients, and business users
- ✓ Handoff documentation delivered

Glossary of AWS Terms

A quick-reference list of every AWS term used in this guide, in plain language.

Term	Plain-language meaning
AWS	Amazon's cloud platform — rented computers, storage, and networking
Cloud hosting	Running an application on a remote, always-on computer instead of a local one
Region	A physical AWS data-center location (e.g. N. Virginia, Mumbai)
IAM user	A restricted login for an AWS account, safer than the master "root" login
EC2	A virtual server rented from AWS that runs your application
AMI	A template operating system image used to create an EC2 server (e.g. Ubuntu)
Instance type	The "size" of a virtual server — how much processing power and memory it has
Key pair	A digital lock-and-key used to securely log in to a server instead of a password
Security Group	A virtual firewall controlling which network traffic can reach a server
Port	A numbered "door" a server uses for a specific type of traffic (e.g. 443 for HTTPS)
SSH	A secure method for opening a command-line session on a remote server
Elastic IP	A fixed, permanent public address for an EC2 server
DNS	The system that matches readable domain names to numeric IP addresses
Route 53	AWS's DNS service, used to manage domain names
Hosted Zone	A container in Route 53 that holds all DNS records for one domain
A Record	A DNS instruction that points a domain name to a specific IP address
Domain / Subdomain	A readable web address / a named branch under that address
SSL / TLS	Technology that encrypts data traveling between a browser and a server
HTTPS	The secure, encrypted version of the web's HTTP protocol
ACM	AWS Certificate Manager — issues free SSL/TLS certificates
Uvicorn	The program that runs a FastAPI Python application and serves web requests

Executive Summary

This page summarizes the Invoice Workbench AWS deployment for readers who want the outcome, not the technical detail.

Required Client Inputs

- AWS account access (IAM user)
- Chosen hosting URL/domain strategy

Required AWS Services

- EC2 — the server that runs the app
- Elastic IP — a permanent address
- Route 53 — the domain name system
- SSL/HTTPS — encryption & trust

Deployment Flow

- Server created and secured
- Invoice Workbench installed and tested
- Permanent address attached
- Domain name connected
- Security certificate enabled

Final Hosted URL

`https://invoiceworkbench.company.com`

Business Benefits

Availability	Invoice Workbench is available 24/7 from any device, not just one laptop.
Professionalism	A branded, memorable web address instead of a raw IP or local file path.
Security	All traffic is encrypted, protecting sensitive invoice data.
Scalability	Capacity can grow as usage grows, without rebuilding the application.

BOTTOM LINE

With the two client inputs above, Invoice Workbench can move from a single developer's laptop to a secure, always-available, professionally branded web application.